AI Threat Modeling: OWASP LLM Top 10, Adversarial ML, and Practical Mitigations

Threat modeling for AI systems asks the same four questions as classic threat modeling — *What are we building? What can go wrong? What are we going to do about it? Did we do a good job?* — but over a system whose trust boundaries are unusual: **natural-language input is also control flow**, retrieved data can carry instructions, and the model is a non-deterministic, partly-attacker-influenced component.

Modeling approach

1. **Diagram data and trust flows.** Mark every place untrusted text enters the model's context: user input, RAG/retrieval, tool outputs, prior messages, MCP tool descriptions, system-prompt assembly. Each is an injection ingress.
2. **Identify the model's blast radius.** Enumerate every tool/action the model can trigger and the data each can read or mutate. This is the "excessive agency" surface (OWASP LLM06).
3. **Apply STRIDE + AI-specific lenses.** Augment STRIDE with the OWASP LLM Top 10 and MITRE ATLAS tactics so you cover poisoning, extraction, and evasion that STRIDE alone misses.
4. **Rank by realistic impact.** An indirect injection that reaches a money-moving tool outranks a jailbreak that only produces disallowed text.

OWASP Top 10 for LLM Applications (2025)

ID	Risk	One-line threat
LLM01	Prompt Injection	Crafted input (direct or via retrieved content) subverts instructions.
LLM02	Sensitive Information Disclosure	Model leaks PII, secrets, or proprietary data.
LLM03	Supply Chain	Compromised models, datasets, plugins, or MCP servers.
LLM04	Data and Model Poisoning	Tampered training/fine-tune/RAG data alters behavior.
LLM05	Improper Output Handling	Downstream system trusts model output (XSS, SSRF, RCE).
LLM06	Excessive Agency	Model has too much permission/autonomy; actions run unchecked.

LLM07	System Prompt Leakage	Hidden instructions/secrets in the system prompt are exposed.
LLM08	Vector and Embedding Weaknesses	RAG/embedding store poisoning, inversion, cross-tenant leakage.
LLM09	Misinformation	Confident, wrong output; over-reliance without verification.
LLM10	Unbounded Consumption	Resource/cost exhaustion, denial of wallet, model DoS.

Adversarial ML attack techniques

Prompt injection (LLM01 / ATLAS AML.T0051). Direct (`AML.T0051.000`) places the payload in user input; indirect (`AML.T0051.001`) hides it in content the model retrieves (a web page, a document, an email, another tool's output). Indirect is the more dangerous variant because the attacker need not be the user — they only need to control something the agent will read.

Jailbreaking (ATLAS AML.T0054). Persona overrides ("you are DAN"), hypothetical framing, refusal suppression, and payload splitting push the model past its safety policy. Jailbreaks target the *policy* layer; injections target the *instruction* layer. They compose.

Data poisoning (LLM04 / ATLAS AML.T0020). An attacker who can influence training data, a fine-tune set, or — most accessibly — a RAG index, plants false "facts" or latent instructions/backdoors that surface at inference time.

Model extraction / inversion (LLM02, LLM07 / ATLAS AML.T0024, T0056). Querying the API to steal the system prompt, recover memorized secrets/training data, or clone model behavior.

This repo's `airte.redteam` package implements runnable cases for each of these, and `airte.atlas` maps them to ATLAS tactics so you can see coverage per kill-chain stage.

Mitigations that hold up in production (not just on paper)

The uncomfortable truth: **there is no reliable, complete prompt-injection filter.** Heuristic input classifiers (this repo ships one) reduce noise but are bypassable. Production resilience comes from *architecture*, not detection:

- **Privilege separation / least agency (defeats LLM06).** The model proposes; a deterministic, permission-checked layer disposes. Scope every tool to the *end user's* entitlements, not the service account's. See `airte.guardrails.AgentGuard`.
- **Treat retrieved content as data, never instructions (defeats indirect LLM01).** Spotlight/fence retrieved chunks and tag them as untrusted. See `airte.guardrails.rag_guardrails.sanitize_document`.

- **Output handling (defeats LLM05).** Never `eval`/render/exec model output; encode it for its sink; validate against a schema. Redact secrets on the way out (`airte.guardrails.output_filters`).
- **Human-in-the-loop for irreversible actions.** Approval gates on money/data/identity changes contain both bugs and successful attacks.
- **Sensitive-data minimization (defeats LLM02/LLM07).** Keep secrets out of system prompts and context; pull them at execution time from a vault. DLP-scan anything entering or leaving the boundary (`airte.data_pipeline.dlp`).
- **Rate/cost limiting (defeats LLM10).** Cap tokens, request size, tool-call count, and per-principal spend at the gateway (`airte.cloud.api_gateway`).
- **Supply-chain controls (defeats LLM03).** Pin and verify models, datasets, and MCP servers; scan dependencies; review tool descriptions as code.

"Did we do a good job?" — continuous validation

Threat modeling is not a one-time artifact. Convert each identified threat into a red-team test case and run it in CI. `airte.redteam.harness` gives you an attack-success-rate (ASR) metric you can gate on, so regressions in your guardrails fail the build the same way a failing unit test would.